

Outline:

- Introduction to LLM interactions using Kuberentes
- Basics of Model Context Protocol (MCP)
- Creating and testing MCP servers using Python and FastMCP
- Limitations and threats to MCP servers

Study environment:

- Lab materials are present on Moodle
- We will use AWS Academy resources:
 - https://www.awsacademy.com/vforcesite/LMS_Login
- In our laboratory we will use :
 - aws-cli - <https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2-linux.html>
 - kubectl - <https://kubernetes.io/docs/tasks/tools/>

When you execute all commands from the command line with the version argument, they should not exit with an error.

Before ending the class :

- Please combine the results of all tasks in one single text file and attach it to UPEL (at the end of our meeting)
- At the end of lab:
 - Click the Reset button on Vocareum labs (not End Lab)

Task -1 – Prepare and Verify your environment

First, using the AWS Console, create a cluster in Elastic Kubernetes Service (in the search dialog, type EKS)

Step 1 Configure cluster

Step 2 Specify networking

Step 3 Configure observability

Step 4 Select add-ons

Step 5 Configure selected add-ons settings

Step 6 Review and create

Configure cluster

Configuration options - new [info](#)

Choose how you would like to configure the cluster.

☐ Quick configuration (with EKS Auto Mode) - new
Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

☒ Custom configuration
To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode - new [info](#)

Choose if you would like to use EKS's Auto Mode.

☒ Use EKS Auto Mode
EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

► Included capabilities

Cluster configuration [info](#)

Name
Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

suu-cluster-1

The cluster name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Cluster IAM role [info](#)
Select the Cluster IAM role to allow the Kubernetes control plane to manage AWS resources on your behalf. This cannot be changed after the cluster is created. To create a new custom role, follow the instructions in the [Amazon EKS User Guide](#).

LabRole [Create recommended role](#)

Kubernetes version settings

Kubernetes version [info](#)
Select Kubernetes version for this cluster.

1.31

Upgrade policy [info](#)
Choose one of the following options. You can switch the setting later while the standard support period is in effect.

☒ Standard
This option supports the Kubernetes version for 14 months after the release date. There is no additional cost. When standard support ends, your cluster will be auto upgraded to the next version.

☐ Extended
This option supports the Kubernetes version for 26 months after the release date. The extended support period has an additional hourly cost that begins after the standard support period ends. When extended support ends, your cluster will be auto upgraded to the next version.

- Step 1
Configure cluster
- Step 2
Specify networking
- Step 3
Configure observability
- Step 4
Select add-ons
- Step 5
Configure selected add-ons settings
- Step 6
Review and create

Specify networking

Networking [info](#)

IP address family and service IP address range cannot be changed after cluster creation.

VPC [info](#)

Select a VPC to use for your EKS cluster resources.

vpc-027fc650f81ac161a | Default

Subnets [info](#)

Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets

subnet-06edb215982ca0285
us-east-1a 172.31.80.0/20

subnet-0d8dcda42ec3d0ba7
us-east-1b 172.31.16.0/20

Clear selected subnets

Additional security groups - optional [info](#)

EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose additional security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups

Choose cluster IP address family [info](#)

Specify the IP address type for pods and services in your cluster.

☒ IPv4

☐ IPv6

Configure Kubernetes service IP address block [info](#)

☐ Specify the range from which cluster services will receive IP addresses.

Configure remote networks to enable hybrid nodes - new [info](#)

EKS Hybrid Nodes enables you to use on-premises and edge infrastructure as nodes in EKS clusters.

☐ Specify the CIDR blocks for your on-premises environments that you will use for hybrid nodes.

Cluster endpoint access [info](#)

Configure access to the Kubernetes API server endpoint.

☐ Public

The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.

☒ Public and private

The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.

☐ Private

The cluster endpoint is only accessible through your VPC. Worker node traffic to the endpoint will stay within your VPC.

► Advanced settings

Cancel

Previous

Next

Once it is done, add nodes to the cluster. In the cluster options, add nodes to the node group.

[EKS](#) > [Clusters](#) > suu-cluster

Cluster info [info](#)

Status

Active

Kubernetes version [info](#)

1.29

Support type

Standard support until March 23, 2025

Provider

EKS

Overview

Resources

Compute

Networking

Add-ons

Access

Observability

Upgrade insights

Update history

Tags

Nodes (0) [info](#)

Filter Nodes by property or value

Node name

Instance type

Node group

Created

Status

No Nodes

This cluster does not have any Nodes, or you don't have permission to view them.

Node groups (0) [info](#)

Edit

Delete

Add node group

Group name

Desired size

AMI release version

Launch template

Status

No node groups

This cluster does not have any node groups.

Nodes that are not part of an Amazon EKS managed node group are not shown in the AWS console.

Add node group

Step 1

Configure node group

Step 2

Set compute and scaling configuration

Step 3

Specify networking

Step 4

Review and create

Configure node group info

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name
Assign a unique name for this node group.

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role info
Select the IAM role that will be used by the nodes. To create a new role, go to the [IAM console](#).

[Create recommended role](#)

The selected role must not be used by a self-managed node group as this could lead to a service interruption upon managed node group deletion.

[Learn more](#)

Launch template info

These properties cannot be changed after the node group is created.

☒ **Use launch template**
Configure this node group using an EC2 launch template.

Step 1

Configure node group

Step 2

Set compute and scaling configuration

Step 3

Specify networking

Step 4

Review and create

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type info
Select the EKS-optimized Amazon Machine Image for nodes.

Capacity type
Select the capacity purchase option for this node group.

Instance types info
Select instance types you prefer for this node group.

vCPU: 2 vCPUs Memory: 8 GiB Network: Up to 5 Gigabit Max ENI: 3 Max IPs: 36

Disk size
Select the size of the attached EBS volume for each node.
 GiB

Node group scaling configuration

Desired size
Set the desired number of nodes that the group should launch with initially.
 nodes
Desired node size must be greater than or equal to 0

Minimum size
Set the minimum number of nodes that the group can scale in to.
 nodes
Minimum node size must be greater than or equal to 0

Maximum size
Set the maximum number of nodes that the group can scale out to.
 nodes
Maximum node size must be greater than or equal to 1 and cannot be lower than the minimum size

Next, provision the cluster. First, set up an AWS account on your local machine. Please perform the following steps:

- You can create environment by typing *aws configure*
 - Add at least one data to config
- Copy your AWS credentials from Vocareum site to local machine. Store them in `~/.aws/credentials`.
- Create default values for session:

```
aws configure set default.region us-east-1
aws configure set default.output table
```
- Check the environment (there should be no ec2 instances and no errors on console):

```
aws sts get-caller-identity
```

Next, configure the *kubectl* command

```
aws eks describe-cluster --region us-east-1 --name <your cluster> --query 'cluster.status' --output text
```

```
aws eks --region us-east-1 update-kubeconfig --name <your cluster>
```

You should see the info that the cluster is ready. Now check the status of nodes:

```
kubectl get nodes
```

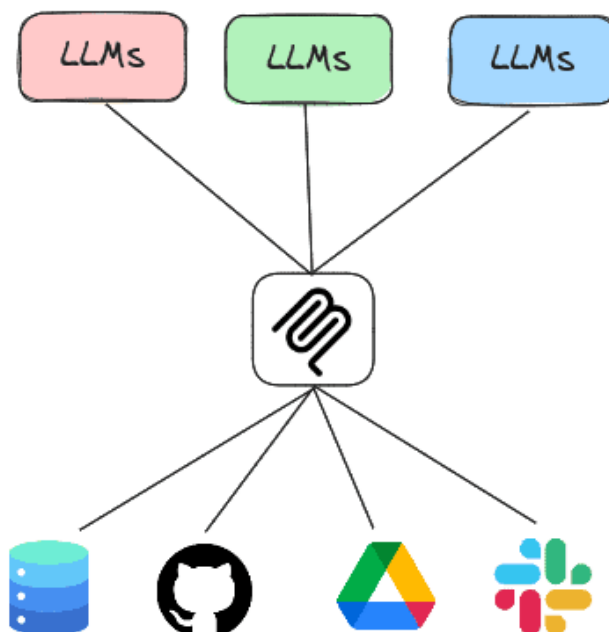
Also, check that all Kubernetes components are ready (here we are using *kube-system* namespace):

```
kubectl --namespace kube-system get pods
```

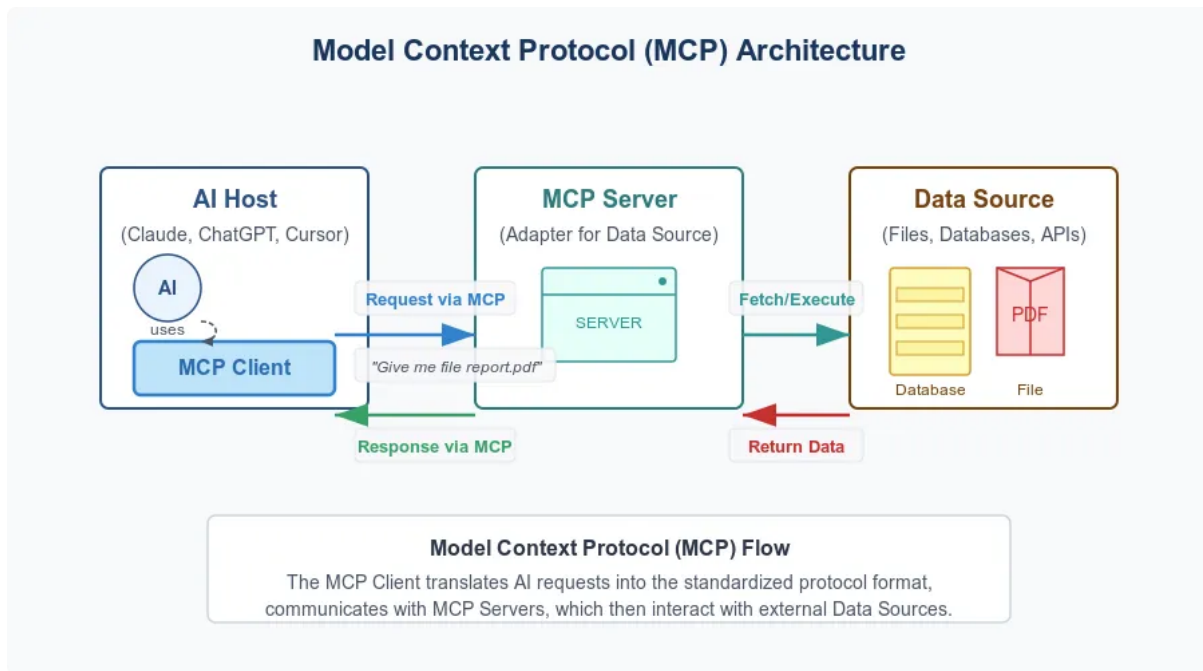
TASK -1 – list all nodes and components in your kube-system namespace

Task 0 - Introduction to Model Context protocol

The Model Context Protocol (MCP) is a standard protocol that enables LLMs to retrieve and invoke both local and remote tools to access services. Rather than having every LLM agent implement all the tools it needs to execute, the agent can instead invoke tools implemented by MCP servers that are running either locally or remotely.



MCP communication is built on JSON-RPC 2.0, which provides a common envelope for messages exchanged among MCP hosts, clients, and servers. In practice, operations such as initialization, capability negotiation, tool discovery, resource access, prompt listing, and tool execution are represented as JSON-RPC requests, responses, or notifications. This is useful because the protocol keeps the interaction model independent of any particular programming language or API framework: The current MCP specification describes its data layer as JSON-RPC-based communication among hosts, clients, and servers.



MCP separates this data layer from the transport layer, allowing the same JSON-RPC messages to be carried in different ways. The current specification defines two standard transports: *stdio* and *Streamable HTTP*. With *stdio*, the MCP client launches the server as a local subprocess; the server reads JSON-RPC messages from standard input and writes valid MCP messages to standard output, while standard error can be used for logging. This mode is simple and well suited to local developer tools or IDE integrations. *Streamable HTTP* is better suited to remote or cloud-native deployments: clients send JSON-RPC messages over HTTP POST, and the server may respond either with JSON or with a text/event-stream response for streaming behavior.

Task 0 - Setting up LLM on cluster (simple way)

First, create the required infrastructure using the deployment files attached to the lab

```
kubectl apply -f infra/ollama.yaml
kubectl apply -f infra/ollama-web-ui.yaml
```

Verify that all pods are ready and running

```
kubectl -n mcp-lab get pods
```

Next, using port forwarding, create a direct connection from your local OS to the new pods (we will need two terminals)

```
kubectl -n mcp-lab port-forward svc/open-webui 8080:8080
kubectl -n mcp-lab port-forward svc/ollama 11434:11434
```

Next, install the model and verify that it works (use another terminal)

```
curl http://localhost:11434/api/pull -d '{"name":"qwen3:0.6b"}'
curl http://localhost:11434/api/generate \
  -d '{
    "model": "qwen3:0.6b",
    "prompt": "Explain Kubernetes pods in one short sentence.",
    "stream": false
  }'
```

You can also open a browser on the URL <http://localhost:8080/> and verify that the model is working

TASK 0 – Once the cluster is up, list all pods in the mcp-lab namespace

Task 1 – Create a calculator MCP and import it to Ollama Web UI

Next, let's deploy and debug a sample MCP server. In the example, we use new Kubernetes objects. A ConfigMap is a Kubernetes object used to store non-confidential configuration data separately from the application container image. It can contain key-value pairs, configuration files, or command-line settings that are later mounted into Pods as environment variables, files, or arguments. This allows the same application image to be reused across environments such as development, testing, and production without rebuilding the container.

Internally, a ConfigMap is stored as an API object in the Kubernetes control plane. When you create a ConfigMap, it is sent to the Kubernetes API server, validated, and persisted in etcd, the key-value store Kubernetes uses to maintain cluster state. When a Pod uses a ConfigMap, the kubelet on the target node retrieves the required data from the API server and makes it available to the container, for example, as environment variables or mounted files. For sensitive data such as passwords, tokens, or private keys, use Kubernetes Secrets rather than ConfigMaps.

First deploy calculator MCP on Kubernetes cluster:

```
kubectl apply -f mcp-impl/calculator-mcp.yaml
```

you will need to check the logs use following command:

```
kubectl -n mcp-lab logs deployment/calculator-mcp
```

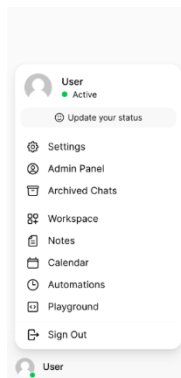
Next, using the port-forwarding mechanism, verify that the mcp is working

```
kubectl -n mcp-lab port-forward svc/calculator-mcp 8000:8000
```

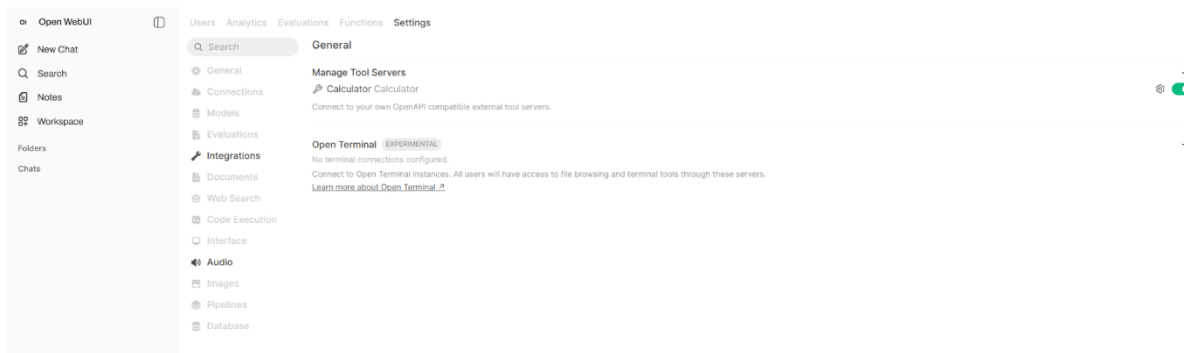
In the new terminal session type:

```
curl -s http://localhost:8000/multiply \
  -H "Content-Type: application/json" \
  -d '{"a": 17, "b": 23}' | jq
```

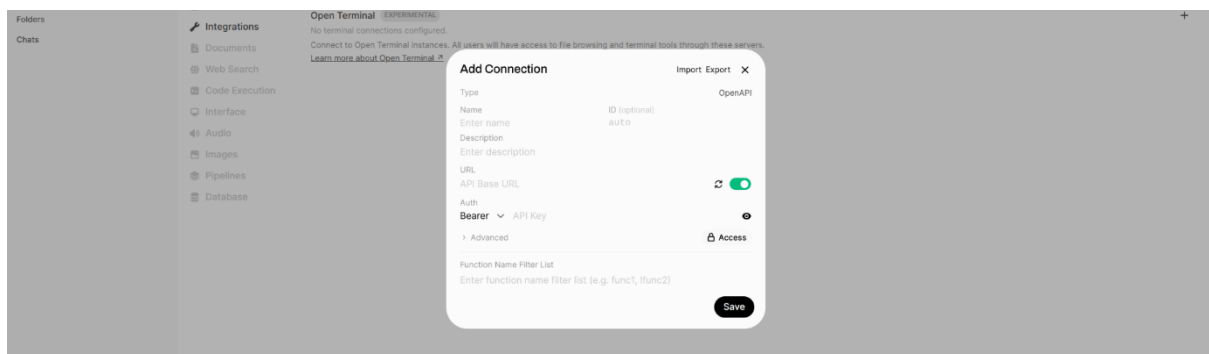
Next, integrate mcp with the Ollama Web UI. First, select Admin Panel from the user preferences:



Next, open the settings integrations dialog:

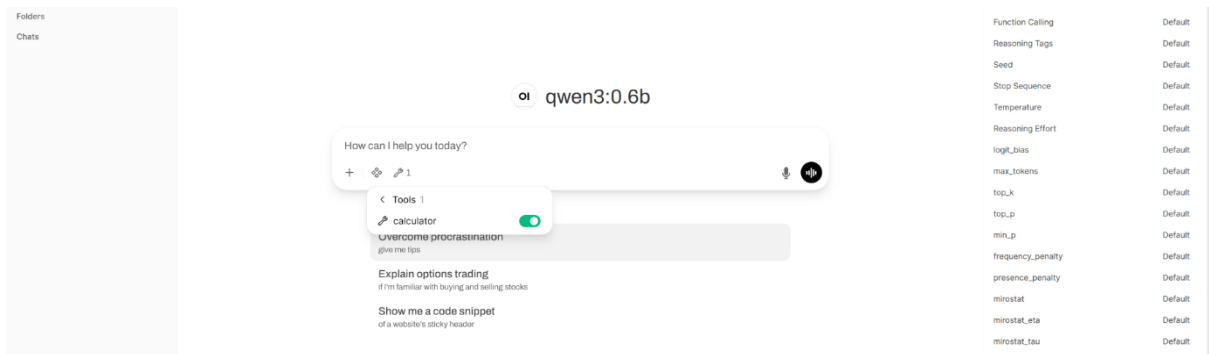


Select add connections, then import calculator-description.json

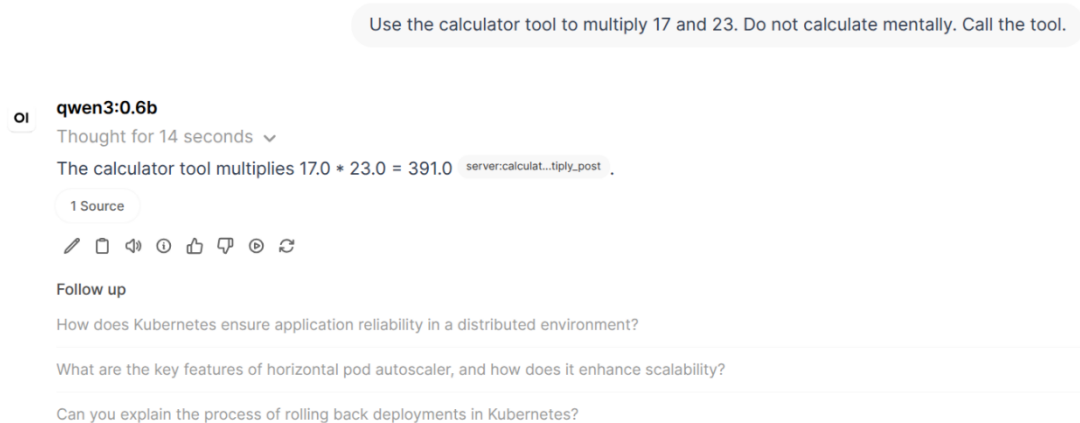


Verify that there's connectivity to the mcp server and that the import is successful (green icon).

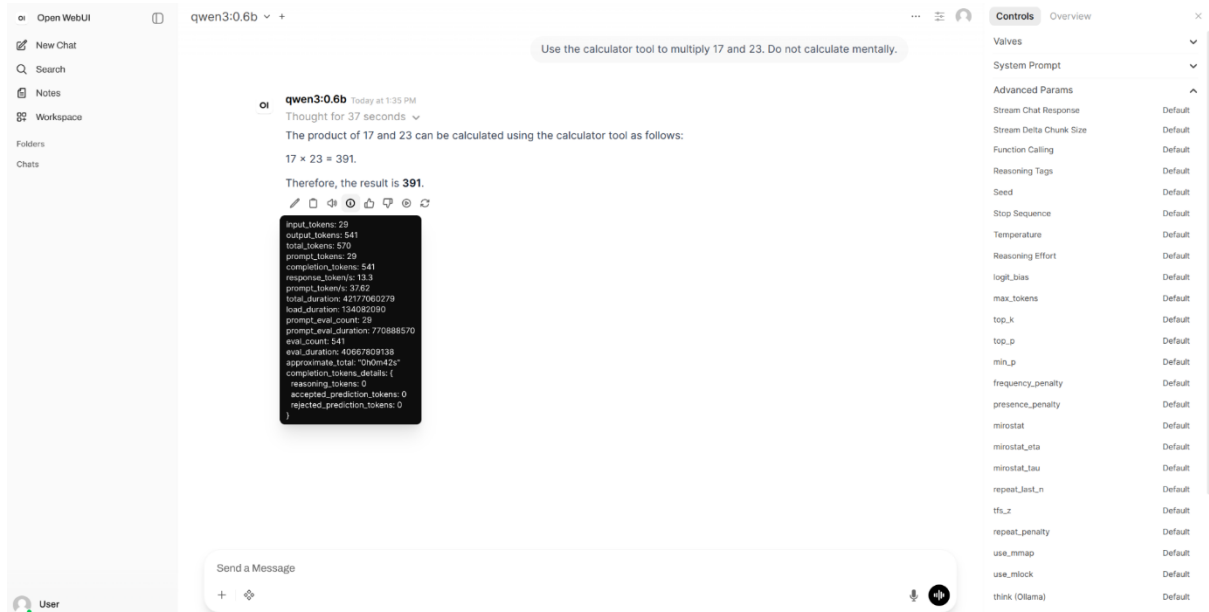
Next, open a new chat dialog and enable the tool in the chat window (select integrations):



Next, perform the calculations in the chat.



Create two chats, one with the tool and one without. Try to multiply 17 and 23. In order to make sure that the tool is invoked, use: "Use the calculator tool to multiply 17 and 23. Do not calculate mentally." Below the chat window, you can see how many tokens it consumes. Next, compare the costs:



TASK 1 – Compare and provide Input tokens, Output tokens, Total tokens, Duration for both options

Task 2 – Inspect Calculator MCP server

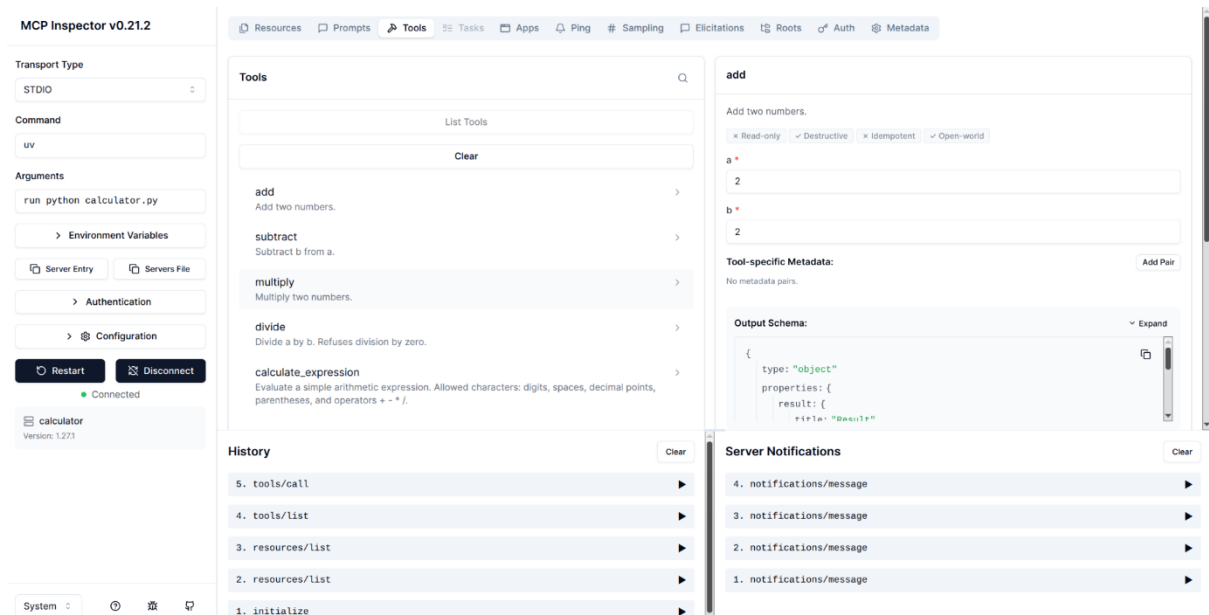
You can debug and analyze your MCP server's code using the MCP Inspector tool (<https://github.com/modelcontextprotocol/inspector>). Navigate to the website and install it on your machine. To access the implementation, please copy the code from the deployment file to a separate file, calculator.py, and start the debugging session.

```
uv init
uv add fastmcp
```

Next, execute the tool:

```
npx @modelcontextprotocol/inspector uv run python calculator.py
```

You can test how it works in your browser



TASK 2 – add screenshot with MCP inspector view for calculator MCP

Task 3 – Create a Kubernetes MCP server

Based on the calculator example, create a sample Kubernetes MCP server that will take information about the pods in your environment. Implement the missing methods and provide the following functionality: list pods, describe a pod, and get logs from a pod.

TASK 3 – Paste code from your implementation of Kubernetes mcp, add screenshots with responses and costs

Task 4 – Prompt injection attacks

Extend the Kubernetes MCP server to support some unsafe operations. Perform a prompt injection attack on the infrastructure

TASK 4 – Paste code from your implementation, add screenshots with responses and costs

Note: Issues

One of the most important issues with the MCP is the expansion of context, which may cause issues with the reasoning process¹

¹ <https://github.com/chroma-core/context-rot>

